

TILEGRAD: a typed tile language over TINYGRAD UOps

North-star specification, Version 1

TileGrad contributors

July 2026

Abstract

TILEGRAD is a Python-embedded language for tiled compute kernels. Programs declare tile shapes, memory placement, launch shape, and coarse scheduling intent; the compiler owns lane-level distribution, synchronization, and legal instruction selection. TILEGRAD lowers into TINYGRAD UOps and delegates backend code generation, compilation, and runtime execution to TINYGRAD.

This is a *north-star contract*, not a snapshot of the current experimental implementation. It states what TILEGRAD must become to be considered done, which decisions are stable, and which surface is explicitly deferred. Normative text uses the keywords **MUST**, **MUST NOT**, **SHOULD**, and **MAY**. Exact keyword signatures, error-message wording, and numerical tolerances live in the API reference and the conformance test suite respectively; this document defers to them where it says so.

Contents

1	Why TILEGRAD	1
2	Target workload	1
2.1	Canonical GEMM	1
2.2	FlashAttention forward	2
3	Program contract	3
3.1	Value categories	3
3.2	Outputs	4
3.3	Sizes, ranks, and forms	4
4	Type and effect system	4
4.1	Memory spaces	4
4.2	Views and bounds	4
4.3	Effects and synchronization	4
4.4	Uniformity	5
4.5	Scalar types	5
5	Operation categories	5
5.1	Loops	5
5.2	Logical parallel mapping	5
5.3	Collective tile operations	6
5.4	Fragment algebra	6
5.5	Scalar memory	6
6	Capabilities	7
7	Lowering contract	7
7.1	Differentiation	8
8	Done criteria	8
9	Deferred, with reasons	9

1 Why TILEGRAD

TINYGRAD compiles tensor graphs and runs them, but it has no language for an *author to write a custom kernel*. Today the only way to add one is to hand-roll bare UOps: lane indices, shared-memory barriers, swizzled layouts, and WMMA operand scopes are all the user’s problem. Triton and TileLang solve this in general, but they also own their own backends, schedulers, and autotuners; they are large systems to bridge to TINYGRAD from and large systems to keep coherent.

TILEGRAD takes a narrower position:

- Make tiled kernels substantially easier to write than raw TINYGRAD custom UOps, without inventing a new backend.
- Keep tile shape, memory placement, launch shape, and coarse scheduling intent visible in source and in debug IR.
- Provide one correctness path whose meaning is independent of target instruction selection; a scalar or SIMT fallback is always semantically equivalent to any selected native capability.
- Permit optimized lowerings without changing source-level semantics.

TILEGRAD owns the typed tile IR, view and effect analysis, uniformity, canonical logical layouts, synchronization planning, tile-op legalization, and SIMT algorithms. TINYGRAD owns UOps, scalar and vector rendering, device-specific code generation, compilation, launch, and outer tensor-graph scheduling. The compiler boundary between the two is the UOp interface described in Section 7.

2 Target workload

The motivating workload is tiled GEMM and FlashAttention-style fused attention. If TILEGRAD cannot express a clean FlashAttention forward pass, its abstractions are wrong. The canonical GEMM in Section 2.1 and the FlashAttention sketch in Section 2.2 are part of the contract, not illustrations.

2.1 Canonical GEMM

The following spelling is the canonical V1 frontend example. The frontend is a Python JIT decorator; calling a decorated function specializes, compiles, caches, and executes the kernel. Eager or lazy evaluation of the body is not inferred from the return type.

```
@tilegrad.jit
def matmul(A, B, *,
           BM: T.Const[int] = 64,
           BN: T.Const[int] = 64,
           BK: T.Const[int] = 32):
    M, N, K = T.const("M", "N", "K")
    A: T.Tensor[(M, K), T.float16]
    B: T.Tensor[(K, N), T.float16]
    C = T.empty((M, N), T.float16)

    with T.Kernel(T.ceildiv(N, BN), T.ceildiv(M, BM),
                  threads=128) as (bx, by):
        A_shared = T.shared((BM, BK), T.float16)
        B_shared = T.shared((BK, BN), T.float16)
        accum = T.fragment((BM, BN), T.float32)
        T.clear(accum)

        for ko in T.Pipelined(T.ceildiv(K, BK), stages=3):
            T.copy(A.tile((by*BM, ko*BK), (BM, BK)), A_shared, pad=0)
            T.copy(B.tile((ko*BK, bx*BN), (BK, BN)), B_shared, pad=0)
            T.mma(A_shared, B_shared, accum)

        T.copy(accum, C.tile((by*BM, bx*BN), (BM, BN)))
    return C
```

The reader who only reads this example should already understand the model: runtime tensors are declared with shape and dtype; specialization symbols are named once and bind to runtime dimensions; the launch region yields grid coordinates; managed shared and collective fragment allocations live inside that region; tile ops (`T.copy`, `T.clear`, `T.mma`) exchange typed tile regions; pipelining is a hint.

2.2 FlashAttention forward

The same abstractions must cover the online-softmax FlashAttention forward pass. The sketch below is part of the spec; a conformance kernel implementing it is required (Section 8).

```
@tilegrad.jit
def flash_fwd(Q, K, V, *,
              Bq: T.Const[int] = 64,
              Bk: T.Const[int] = 64):
    B, Sq, Sk, D = T.const("B", "Sq", "Sk", "D")
    Q: T.Tensor[(B, Sq, D), T.float16]
    K: T.Tensor[(B, Sk, D), T.float16]
    V: T.Tensor[(B, Sk, D), T.float16]
    O = T.empty((B, Sq, D), T.float16)
    Lse = T.empty((B, Sq), T.float32)      # log-sum-exp, for backward

    scale = (1.0 / D) ** 0.5

    with T.Kernel(B, T.ceildiv(Sq, Bq)) as (bb, qg):
        q_tile = T.shared((Bq, D), T.float16)
        k_tile = T.shared((Bk, D), T.float16)
        v_tile = T.shared((Bk, D), T.float16)
        s_frag = T.fragment((Bq, Bk), T.float32)
        acc = T.fragment((Bq, D), T.float32)
        m_run = T.fragment((Bq, 1), T.float32) # running row-max
        l_run = T.fragment((Bq, 1), T.float32) # running row-sum
        o_run = T.fragment((Bq, D), T.float32) # running output

        T.clear(acc); T.clear(m_run); T.clear(l_run); T.clear(o_run)
        T.copy(Q.tile((bb, qg*Bq, 0), (Bq, D)), q_tile)

        for ko in T.Pipelined(T.ceildiv(Sk, Bk), stages=2):
            T.copy(K.tile((bb, ko*Bk, 0), (Bk, D)), k_tile)
            T.copy(V.tile((bb, ko*Bk, 0), (Bk, D)), v_tile)

            T.clear(s_frag)
            T.mma(q_tile, k_tile, s_frag)      # (Bq,D) @ (D,Bk)
            s_frag = s_frag * scale

            m_blk = T.reduce_max(s_frag, axis=-1, keepdim=True)
            m_new = T.maximum(m_run, m_blk)
            alpha = T.exp2(m_run - m_new)      # exp2 base for exp/log2
            beta = T.exp2(s_frag - m_new)

            l_run = l_run * alpha + T.reduce_sum(beta, axis=-1, keepdim=True)
            o_run = o_run * alpha
            T.mma(beta, v_tile, o_run)        # accumulate V contributions
            m_run = m_new

        o_run = o_run / l_run
        T.copy(o_run, O.tile((bb, qg*Bq, 0), (Bq, D)))
        T.copy(m_run + T.log2(l_run),
              Lse.tile((bb, qg*Bq), (Bq,)))
    return O, Lse
```

This example exercises every operation category TILEGRAD commits to: specialization symbols shared across multiple tensors, a 2D grid, managed shared staging, collective fragments, elementwise fragment algebra, row reductions

with keepdim broadcasting, scalar-fragment mixing, two MMAs per inner loop, a pipelined loop, and multi-output return. If any of those is missing or awkward, the spec has a hole.

3 Program contract

A decorated function consists of:

1. runtime tensor and scalar parameters;
2. keyword-only compile-time parameters (`T.Const`);
3. tensor and output declarations;
4. exactly one `with T.Kernel(...)` launch region;
5. zero or more compile-time helper or macro expansions; and
6. a return of every declared output, preserving tuple order.

3.1 Value categories

Category	Meaning	Permitted uses
Compile-time constant	Included in the specialization cache key.	Python control flow, tile and allocation shapes, workgroup shape, layouts, unrolling, and all device expressions.
Tensor shape constant	Inferred from a runtime tensor, then specialized.	Grid extents, output shapes, bounds, and device expressions.
Runtime scalar	Passed to the device program without specialization.	Device expressions, predicates, indices, and loop bounds; not launch, allocation, fragment, layout, unroll, or workgroup shapes.

Tensor rank and dtype are always specialization-time information. `T.const(...)` declares named specialization symbols; each symbol must occur as a dimension in at least one input tensor declaration, and repeated occurrences must agree at specialization. An unbound or conflicting symbol is a specialization error. Non-constant control flow and non-constant workgroup shapes are rejected before device execution.

3.2 Outputs

`T.empty(shape, dtype)` declares a dense, contiguous global output allocated by the host wrapper before launch. Every output **MUST** be returned exactly once, in source order, as a tuple when there are several. Outputs must not alias one another or any input. Runtime tensor inputs are read-only in V1; an in-place extension would need to make aliasing explicit. Scalar device results are represented as rank-zero or one-element output tensors, not Python values.

3.3 Sizes, ranks, and forms

Core implementations **MUST** support tensor ranks zero through four and one- to three-dimensional grids and workgroups. External tensors are dense, non-overlapping, and contiguous, matching the current TINYGRAD custom-kernel boundary. Higher ranks and non-contiguous external tensors are capabilities.

4 Type and effect system

4.1 Memory spaces

Space	Ownership	Shape	Purpose
Global	Invocation/tensor	Static rank; specialized dimensions	Inputs and outputs.
Shared	Workgroup	Compile-time constant	Cooperative staging and communication.
Fragment	Workgroup collective	Compile-time constant	Logically shaped values distributed across lanes.
Private	One local agent	Compile-time constant	Scalars and statically indexed register tuples.

Shared, fragment, and private allocations have lexical lifetime inside one launch region. Dynamic shared allocation is outside V1. Private tuple indices must become constant after specialization and unrolling; otherwise compilation fails rather than silently spilling to another memory space.

4.2 Views and bounds

A buffer type is the tuple

(dtype, rank, shape, strides, address space, access).

A view is

$$V = (B, o, \mathbf{s}, \mathbf{d}, n_B),$$

where B is a base buffer, o an element offset, $\mathbf{s}$ a logical shape, $\mathbf{d}$ element strides, and the base storage contains n_B elements. Coordinate $\mathbf{i}$ is in bounds exactly when

$$\text{inbounds}(V, \mathbf{i}) \iff \left(\bigwedge_k 0 \leq i_k < s_k \right) \wedge 0 \leq \text{addr}(V, \mathbf{i}) < n_B, \quad \text{addr}(V, \mathbf{i}) = o + \sum_k i_k d_k.$$

`tile(origin, shape)` **MAY** intentionally create a logical view extending beyond base storage; masked load, store, fill, and copy use the predicate above to define edge behavior. `subview` and `transpose` are view operations and do not copy. `reshape` is valid only when contiguity can be proven. Writable zero-stride views, overlapping writable views, and negative strides are invalid in V1.

Two views overlap when their sets of valid base-storage addresses intersect. Copy between identical views is a no-op. Every other overlapping source and destination copy is invalid; V1 provides no snapshot or `memmove` semantics, and compilation must fail when disjointness cannot be proven.

4.3 Effects and synchronization

Effects to the same logical location are ordered by source order unless they occur in an unordered parallel region. Two unordered writes, or an unordered read and write without a defined collective or synchronization relation, form a data race and are invalid; V1 has no atomic escape from this rule.

Managed high-level operations carry effect analysis and **MUST** conservatively preserve read-after-write, write-after-read before storage reuse, write-after-write source order, and loop-carried dependencies. A portable workgroup barrier waits for all local agents and establishes a happens-before relation from prior shared writes to subsequent shared reads by participants. Every barrier **MUST** be reached convergently by the same participant set.

Expert code **MAY** request manually synchronized shared storage (`sync="manual"`). Managed and manual synchronization must not be mixed for one allocation. Raw shared access through explicit thread identifiers is permitted only for manually synchronized allocations; the author is then responsible for barriers and race freedom. There is no grid-wide barrier in V1.

4.4 Uniformity

A conditional is *workgroup-uniform* when every local agent evaluates the same branch. Collective tile operations and barriers may appear only in uniform control flow. The compiler **MUST** prove uniformity from compile-time values, grid coordinates, and other known-uniform values, or reject the program. Local thread identifiers and values loaded

under thread-varying indices are conservatively non-uniform. V1 has no user assertion that overrides uniformity analysis.

4.5 Scalar types

Core scalar types are `bool`, signed and unsigned 8-, 16-, 32-, and 64-bit integers, `float16`, `bfloat16`, and `float32`. Additional TINYGRAD dtypes are capabilities. Expressions are statically typed; implicit conversion is limited to a Python literal converted to the type required by its context. All other dtype changes use `T.cast`. Integer indices use the implementation’s index type and **MUST NOT** be silently converted from floating point.

5 Operation categories

This section names the operation categories V1 **MUST** support. Exact kwarg signatures, defaults, and abbreviations live in the API reference; this section defines what each category means and what the compiler guarantees about it.

5.1 Loops

Form	Semantics
Serial	Ordered iterations over a half-open integer interval.
Unroll	Serial semantics with a compile-time request to expand iterations.
Reduce	Ordered recurrence semantics that may reassociate under the numerical contract.
Parallel	An unordered logical Cartesian iteration space; every point executes exactly once.
Pipelined	Serial logical semantics with a positive compile-time pipeline hint.

5.2 Logical parallel mapping

`T.Parallel` is not an alias for a hardware local axis. It defines an unordered logical iteration space distributed over the current workgroup. Let the iteration shape be $(e_0, \dots, e_{r-1})$, let P be the workgroup size, and let $\ell \in [0, P)$ be the row-major linear local-agent identifier. The compiler selects from a bounded set of canonical policies:

- `row_major`: $q_R(\mathbf{i}) = \sum_k i_k \prod_{j>k} e_j$; owner $q_R \bmod P$, private slot $\lfloor q_R/P \rfloor$;
- `column_major`: $q_C(\mathbf{i}) = \sum_k i_k \prod_{j<k} e_j$; owner $q_C \bmod P$, private slot $\lfloor q_C/P \rfloor$; and
- an internal exact mapping required by a selected native matrix capability.

An agent executes the coordinates whose linearized value is $\ell + tP$ for nonnegative t while it is less than $\prod_k e_k$. The source may request a canonical policy or `auto`; `auto` may select only from the specified finite set, and selection **MUST** appear in legalized debug IR. If explicit thread identifiers affect the same memory operations as a `T.Parallel`, the loop policy **MUST** be explicit; an implementation **MUST NOT** make observable behavior depend on an unreported mapping.

5.3 Collective tile operations

Tile operations are collective statements with explicit logical read and write regions. They **MUST** be reached uniformly by every participating local agent; a tile operation in thread-divergent control flow is invalid unless the operation explicitly defines predicated participation.

Fill and clear. $\text{FILL}(D, x)$ writes $\text{cast}_{D.\text{dtype}}(x)$ to every valid logical coordinate of D . $\text{CLEAR}(D) = \text{FILL}(D, 0)$. Fill never writes outside the destination view.

Synchronous copy. For equal-shaped logical regions S and D ,

$$\text{COPY}(S, D, p) : \quad D[\mathbf{i}] \leftarrow \begin{cases} \text{cast}_{D.\text{dtype}}(S[\mathbf{i}]) & \text{inbounds}(S, \mathbf{i}) \\ \text{cast}_{D.\text{dtype}}(p) & \text{otherwise} \end{cases}$$

when $D[\mathbf{i}]$ is in bounds. Source and destination shapes must be equal; copy does not broadcast and does not infer a tile extent from an unrelated operand. Copy carries safe-to-consume semantics: a subsequent dependent high-level tile operation observes completed values. Instruction choice — scalar, vector, SIMT-cooperative, or target-specific transfer — is not observable; a regular copy never exposes a partially completed asynchronous transfer.

Matrix multiply-accumulate. For rank-two operands,

$$C \leftarrow C + \text{cast}_{T_{acc}}(\text{op}_A(A)) \times \text{cast}_{T_{acc}}(\text{op}_B(B)),$$

with $A : (M, K)$, $B : (K, N)$, $C : (M, N)$ after optional transpose, and no implicit clear of C . Core implementations support `float32` SIMT MMA and `float16/bfloat16` inputs with `float32` accumulation; other dtype pairs are capabilities. The selection mode is one of `auto`, `simt`, or an exact native capability name. An exact request **MUST NOT** silently fall back, and `auto` **MUST** preserve the same logical fragment and numerical contract across all selectable paths.

Axis reduction. `REDUCE( $X, a, f, T_{acc}, k$ )` reduces axis a using operation f (`sum`, `max`, or `min`), accumulator dtype T_{acc} , and keep-dimension flag k . Reduction starts from the operation identity unless an explicit initial value is provided. Maximum and minimum default to `nan_policy="propagate"`; an implementation may emulate this policy when native instructions differ. Reduction order is otherwise implementation-defined.

5.4 Fragment algebra

A fragment is a statically shaped, workgroup-collective logical value; its physical register ownership is not observable source state. Access to a fragment coordinate is legal only inside a tile operation or a matching logical parallel region (equal extents and identical resolved policy). Arbitrary fragment indexing by a runtime thread identifier is invalid. SIMT fallback **MUST** implement the same logical coordinate ownership; it **MUST NOT** replicate the complete fragment in every thread.

`MAPUNARY`, `MAPBINARY`, `SELECT`, and `CAST` apply pointwise and produce a fresh fragment. Operands use right-aligned broadcasting: for each aligned axis, extents must be equal or one; missing leading axes and extent-one axes repeat the same source coordinate. The result extent is the maximum of the aligned extents. Row and column broadcasting are not special cases: shapes $(M, 1)$ and $(1, N)$ follow the same rule. Scalar operands are rank-zero fragments. All operations produce a new logical fragment version; source versions are read before destination writes, including syntactically in-place elementwise forms.

5.5 Scalar memory

Scalar load and store carry explicit masks and alternate values:

$$\begin{aligned} \text{LOAD}(V, \mathbf{i}, m, a) &= \begin{cases} V[\mathbf{i}] & m \wedge \text{inbounds}(V, \mathbf{i}) \\ a & \text{otherwise,} \end{cases} \\ \text{STORE}(V, \mathbf{i}, x, m) : & \quad V[\mathbf{i}] \leftarrow x \quad \text{iff } m \wedge \text{inbounds}(V, \mathbf{i}). \end{aligned}$$

Indexing abbreviations ($V[\mathbf{i}]$) are unmasked forms of the same. Floating-point reassociation and reduction order are implementation-defined within the numerical contract; semantic equality is not generally bitwise equality. Detailed tolerance tables live in the conformance test suite.

6 Capabilities

Capabilities describe optional legal lowerings, not alternate language semantics. They are the single mechanism by which a target advertises more than scalar/SIMT fallback. A capability record has:

- a versioned, namespaced name;
- a target predicate;
- operand and accumulator dtypes;
- an atom shape (M_a, N_a, K_a) ;
- a participant count;
- operand scope requirements (e.g., shared or private);
- the internal fragment layout the capability assumes;
- the exact TINYGRAD WMMA argument schema used by the compatibility adapter.

A user requests a capability by name via the operation’s mode argument, e.g. `T.mma(A, B, C, mode="tinygrad.cuda.m16n8k16.f16.f32.v1")`. A backend author publishes one via `tilegrad.capabilities(target)`, which returns records of the following shape:

```
Capability(
  name="tinygrad.cuda.m16n8k16.f16.f32.v1",
  target=("cuda", "sm>=80"),
  operand_dtypes=(float16, float16),
  accumulator_dtype=float32,
  atom=(16, 8, 16),
  participants=32,
  operand_scope=("shared", "shared"),
  layout="row_major_acc",
  wmma_schema=0ps.WMMA(arg_types=(...), ...),
)
```

A logical MMA **MAY** use a native capability only when M , N , and K are integer multiples of the corresponding atom dimensions. The compiler partitions M and N into disjoint atoms, visits K atoms in increasing logical order, and suppresses out-of-bounds stores; the native MMA itself never accesses a partial atom. Renaming or changing the meaning of a record requires a new versioned name; the adapter and record **MUST** be jointly tested against a pinned TINYGRAD revision range.

The compiler **MUST** distinguish an invalid program, a valid program unavailable on the selected profile, a failure to generate valid TINYGRAD UOps, and a backend compilation or runtime failure. Unsupported exact native requests fail during capability legalization, before UOp construction, with a diagnostic stating source construct, expected contract, observed types and shapes, and selected target.

A V1 release **MAY** publish zero required native capabilities; a conforming core profile implementation already runs every program through SIMT fallback. Conformance claims only require the capabilities the implementer advertises under the names they advertise.

7 Lowering contract

Source programs pass through a small number of named stages:

Stage	Responsibility
Specialize	Bind compile-time constants and tensor shape constants.
Typed tile IR	Parse, type-check, verify shapes, effects, access modes, uniformity, and resource rules.
Legalized tile IR	Select canonical logical-parallel and fragment ownership layouts, plan synchronized managed memory, and select SIMT or exact native capabilities.
TINYGRAD UOps	Emit scalar, vector, range, memory, barrier, and optional WMMA UOps; delegate rendering, compilation, launch, and storage.

Each named stage **MUST** be inspectable at the corresponding named IR. An implementation **MAY** add internal passes; their names are not part of the contract.

The interface with TINYGRAD is the UOp set TILEGRAD **MAY** emit. V1 **MUST** emit, and TINYGRAD **MUST** accept:

- scalar ALU, compare, select, and cast nodes;
- LOAD and STORE in global and local address spaces, with explicit mask support;
- LOOP, RANGE, and reduction range nodes with their standard axis-type tags;
- thread and grid coordinate queries;
- BARRIER with the workgroup memory scope; and
- optionally WMMA, schema-defined by an advertised capability.

TINYGRAD **MUST** preserve synchronization structure that TILEGRAD emits: a barrier **MUST** remain a barrier with the same scope and participant set, and TINYGRAD **MUST NOT** reorder a global STORE across a barrier or eliminate a declared output STORE. Within those constraints, TINYGRAD **MAY** fuse, vectorize, lower to native instructions, schedule, and rewrite. TILEGRAD **MUST NOT** rely on TINYGRAD optimization to recover synchronization, fragment ownership, or memory dependences it did not emit.

7.1 Differentiation

V1 has no implicit autograd through a TILEGRAD kernel. Source-level differentiation of a decorated kernel **MUST** fail at JIT specialization with a single clear diagnostic. The supported differentiation paths are:

- an explicit backward kernel written by the same author, returning VJP inputs in source order; and
- a registered VJP rule mapping a forward capability name to a backward capability name through the same capability mechanism.

V1 publishes the VJP registration interface, but **MAY** ship with no registered gradients. The IR **MUST** preserve what an autograd extension would need (input views, output aliasing metadata, and capability names), so post-V1 differentiation is additive, not a rewrite.

8 Done criteria

A V1 release claims *core conformance* by passing, on at least one TINYGRAD backend, the following programs:

1. exact and masked one- through four-dimensional copy;
2. internal strided subviews and transpose;
3. source padding, destination suppression, cross-dtype copy, and rejected overlapping copies;
4. one- through three-dimensional grids, canonical ownership mappings, logical parallel loops, and rejected runtime-scalar launch extents;
5. uniform and rejected divergent collective operations;
6. shared-memory producer/consumer and reuse dependencies;
7. fragment unary, binary, broadcast, select, and cast operations;
8. sum, maximum, and minimum reductions, including non-power-of-two tails;
9. float32 SIMT GEMM with M, N, and K edge tiles;
10. float16 and bfloat16 input GEMM with float32 accumulation;
11. unavailable exact-native capability failures (negative test);
12. multiple output allocation and return ordering; and
13. a small causal and non-causal FlashAttention forward kernel using stable online softmax, edge sequence lengths, reduction, `exp2`, and two MMA operations.

The same mathematical cases **SHOULD** run through SIMT and every advertised native capability; negative tests are part of conformance, not optional robustness. An *accelerated* profile is additive: it **MAY** provide vectorized copy, native matrix capabilities, real software pipelining, hidden asynchronous instructions behind synchronous operations, and additional dtypes. A backend **MAY** implement some accelerated capabilities without implementing all of them.

Numerical tolerances, including a dedicated FlashAttention tolerance row, are normative inputs to the conformance test suite, not this document; they are published alongside it and may be tightened in a patch release. Integer, boolean, movement, mask, and correctly-rounded cast tests are exact.

9 Deferred, with reasons

The following are deferred from V1. Each is listed with a one-line rationale so the deferral can be re-litigated without re-reading the whole spec.

- Automatic schedule search and autotuning: schedule belongs to the author in V1; automatic search is a post-V1 layer over a stable IR.
- Arbitrary layout functions and swizzled layouts: V1 ships a bounded set of canonical logical-parallel and fragment layouts; swizzling arrives with a native capability that needs it.
- TMA, WGMMMA, and TCGEN05: arrive only as named capabilities, never as source constructs.
- TileLang or TVM source compatibility: a different language; reuse is at the user-model level, not the AST level.
- Backend source injection and custom TINYGRAD runtime backends: TINYGRAD owns codegen and runtime.

- Clusters and tensor memory: cooperative multi-workgroup execution is not yet the right surface; arrive with a capability that defines them.
- User-managed asynchronous copy, wait groups, and warp specialization: V1 hides async behind the synchronous copy contract; exposing it directly would break the one-correctness-path rule.
- Scans and atomic memory operations: V1 rejects data races; atomics would need a defined memory-model section first.
- Implicit autograd: see Section 7.
- Grid-wide barriers: not exposed; programs that need them are invalid until a capability defines a legal collective grid reduction.

A Normative IR nodes

The exact in-memory Python classes are not normative. A conforming typed tile IR **MUST** represent at least the following semantic nodes; their names are referenced by the inspectable-stage contract in Section 7.

Node	Operands	Meaning
Program	parameters, outputs, launch	One specialized callable program.
Buffer parameter	type, shape, access	Dense external tensor storage.
Scalar parameter	dtype, category	Compile-time or runtime scalar.
Output specification	shape, dtype, order	Host-allocated returned tensor.
View	base, offset, shape, strides, storage extent	Logical region sharing base storage.
Allocate shared	shape, dtype, sync mode	Workgroup-visible storage.
Allocate fragment	shape, dtype	Collective logical register tile.
Allocate private	shape, dtype	Per-agent statically indexed storage.
Kernel launch	grid, workgroup	Grid of independent workgroups.
Serial loop	start, stop, step, body	Ordered iteration.
Unroll loop	static interval, body	Ordered iteration requested to expand.
Reduce loop	interval, recurrence	Scalar recurrence.
Parallel loop	extents, policy, body	Unordered logical iteration.
Pipelined loop	interval, stages, body	Serial semantics with pipeline hint.
Thread index	dimension	Explicit local-agent coordinate or extent.
If	predicate, branches	Structured control flow.
Load	view, coordinate, mask, alternate	Predicated scalar read.
Store	view, coordinate, value, mask	Predicated scalar write.
Fill	destination, scalar	Collective typed fill.
Copy	source, destination, pad	Collective synchronous region copy.
Fragment map	operation, sources, broadcast shape	Pointwise collective fragment operation.
MMA	A, B, C, transpose, mode	Collective accumulating matrix multiply.
Reduce	source, axis, op, dtype, keepdim	Collective axis reduction.
Barrier	memory scope	Convergent workgroup synchronization.